

TileSight

论文深度解析

A First-Principles Tile-Centric Analytical
GPU Performance Model from Cores to Clusters

核心命题

现代 GPU 程序以 Tile 为编程原语，
性能模型也应以 Tile 为分析原语。

12.35%

单卡 GEMM
pooled MAPE

≈ 1 pp

L2 命中率
平均绝对误差

13.52%

端到端 vLLM
wMAPE

99.66%

Top 5% 候选
达到穷举最优

论文：arXiv:2607.22432v1 | Submitted 24 Jul 2026

作者：Zhiwen Mo et al. | 15 authors

研读重点：Tile 抽象、流水线、缓存复用、跨卡通信、Cost Model、代码与公式审视

整理日期：2026-09-10

00 执行摘要

一句话判断

TileSight 的真正创新不是把 Roofline 做得更复杂，而是让性能分析的抽象层级与现代 GPU 编程的抽象层级对齐：程序员调的是 Tile，模型也直接分析 Tile。

这篇论文提出一个白盒、可组合、跨架构的 GPU 性能模型：将 Tile 内的计算、片上/片外访存和网络通信拆成资源时间向量，再通过依赖 DAG、软件流水线、Wave 调度、Tile 复用距离以及跨设备 Placement 逐层组合，从单个 SM 推到最多 32 张 GPU。论文在 703 个单卡 GEMM 上报告 12.35% pooled MAPE，在 4,680 个 persistent GEMM 上将 L2 命中率误差控制在约 1 个百分点，并在端到端 vLLM serving 上达到 13.52% wMAPE。[1]

对工程实践而言，它最适合充当编译器或自动调优器的第一层 Cost Model：快速排除大量明显较差的 Tile 配置，再把少数候选交给真机 benchmark 和 profiler。论文显示，保留预测最优的 5% schedule，即可在 10 个 LLaMA-derived FP16 GEMM 上达到穷举最优性能的平均 99.66%。[1, Sec. 5.6]

维度	结论
最有价值	Tile 级缓存复用距离 + Placement 统一融合与跨卡通信
最适合	规则、Tile-structured、吞吐主导的 GEMM / Attention / 融合计算通信
不宜直接相信	batch=1、小 M、只有少数 CTA、微秒级 latency-bound kernel 的绝对延迟
关键审视	论文 v1 的拓扑序公式和 SDCM 公式存在值得澄清的数学/排版问题
与你的关联	Full-K deterministic GEMM、MLA decode、MoE/EP、vLLM/SGLang 调优

论文信息与阅读口径

项目	信息
完整标题	TileSight: A First-Principles Tile-Centric Analytical GPU Performance Model from Cores to Clusters
作者	Zhiwen Mo, Yu Cheng, Lei Wang, Zhengju Tang, Lei Xu, Guoyu Li, Yuqi Dong, Lingxiao Ma, Yuqing Xia, Jilong Xue, Fan Yang, Luo Mai, Zhi Yang, Wayne Luk, Hongxiang Fan
机构	Imperial College London、Peking University、Shanghai Jiao Tong University、Tile-AI、Microsoft Research、University of Edinburgh
版本	arXiv:2607.22432v1, 2026-07-24 提交

项目	信息
代码状态	论文称正式发表时开源；DeepStack 仓库提供 source-visible TileSight snapshot，标记为 tilesight-release-2026-07。[2]
本文口径	事实与实验数字以论文 v1 为准；对公式和公开代码的质疑属于独立分析，后续版本可能修订。

阅读地图

- 1. 为什么 Kernel 级 Roofline 无法区分不同 Tile schedule。
- 2. Tile Execution Plan 如何把 Shape、Swizzle、Pipeline、Placement 与多卡映射放进同一个描述。
- 3. 资源时间向量、Prologue-Steady-Epilogue 与 Occupancy 如何共同决定延迟。
- 4. Tile Reuse Distance 如何把遍历顺序映射为 L1.5/L2/HBM 流量。
- 5. 跨设备远程 Tile 如何变成 α - β 通信成本并与本地计算重叠。
- 6. 实验结果究竟证明了什么，以及论文 v1 还有哪些薄弱处。
- 7. 如何把它用在你的小 M Full-K GEMM、MLA、MoE 和 vLLM 调优中。

01 问题背景：为什么需要 Tile 级性能模型

现代 GPU kernel 的性能不再只由总 FLOPs 与总字节数决定。对于同一个数学算子，Tile shape、K 分块、软件流水深度、每个 SM 的驻留 CTA 数、Raster 顺序、Swizzle、中间张量 Placement 以及跨卡 Collective 都可能改变真实执行时间。传统工具各自只能看见其中一部分。

$$T_{\text{roofline}} \approx \max(\text{FLOPs} / \text{Compute Throughput}, \text{Bytes} / \text{Memory Bandwidth})$$

Roofline 给出一阶边界，但无法区分相同 FLOPs / Bytes 下的不同调度。

方法	优势	主要不足	最适用问题
Roofline / GenZ	简单、可解释、快速	资源维度过粗；看不见 pipeline、cache、swizzle 与 wave tail	判断 compute-bound 还是 memory-bound
ML Predictor	特定分布上可能较准	依赖架构数据与训练；跨架构外推和可解释性弱	已知硬件/算子分布内的快速预测
Profiler / NCU	测量接近真实执行	必须先运行；属于事后诊断；instrumentation 可能扰动	回答“刚才为何慢”
Cycle / Trace Simulator	细节丰富	建模和计算成本高；新架构维护困难	研究微架构细节
TileSight	白盒、Schedule-sensitive、可组合	仍是吞吐级近似；需架构校准；小 latency kernel 较弱	运行前筛选配置、诊断 Tile 层瓶颈

抽象错位

Triton、TileLang、CUDA Tile、CuTe/CUTLASS 都在用 Tile 描述程序；如果性能模型仍只接收“总 FLOPs + 总 Bytes”，它就看不见程序员真正能修改的调度变量。

TileSight 因此选择一个中间层：不低到逐条 SASS、逐个 warp，也不高到只剩算子级 FLOPs/Bytes，而是保留 Tile shape、遍历顺序、Action DAG、Pipeline、驻留度、数据 Placement 与分布式映射。这个层级足以表达大量可调优化，同时又比完整模拟器轻量。

论文的核心命题

- 确定性 (Deterministic)：给定 Tile 配置和 Placement，单 Tile 的资源需求可解析计算。
- 可组合 (Composable)：单 Tile 资源向量可以组成 Tile DAG，再组成 Wave 和多设备 Stage。
- 可迁移 (Portable)：Tile 是 A100/Hopper/Blackwell/AMD 等平台共同的程序结构，硬件差异主要进入校准参数。

02 核心抽象：Tile Execution Plan

TileSight 的三级统一建模框架

同一个 Tile 抽象，从单个算子动作一直组合到多 GPU 集群

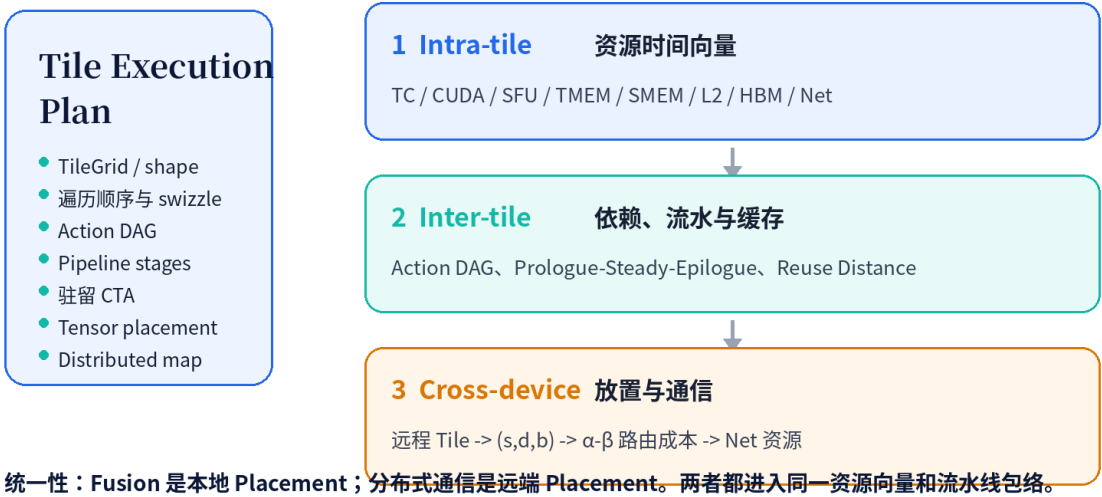


图 1 TileSight 的三级框架。图为本文根据论文结构重绘。

TileSight 的输入不是“一个 GEMM 名称”，而是一份 Tile Execution Plan。它可以抽象写成：

$$P = \{ \text{TileGrid}, \text{TensorAccess}, \text{ActionDAG}, \text{Pipeline}, \text{Placement}, \text{DistributedMap} \}$$

组成	描述内容	典型调优旋钮
TileGrid	全局问题如何切成 $BM \times BN \times BK$ 等 Tile	BM、BN、BK、persistent grid

组成	描述内容	典型调优旋钮
Traversal	CTA / Tile 的发射和遍历顺序	M-major、N-major、panel、Z-order、swizzle
TensorAccess	每个 Tile 读写哪些张量与多少字节	vector width、layout、coalescing
Action DAG	Load、MMA、Reduce、Softmax、Store 等依赖	融合边界、操作顺序、并行 group
Pipeline	stage 数与迭代重叠	double/triple buffer、TMA / async copy
Resident tiles	每个 SM 同时驻留多少 CTA / Tile	寄存器、SMEM、block size
Placement	数据位于 Register/TMEM/SMEM/L2/HBM/远端 GPU	是否落 HBM、是否发生远程访问
DistributedMap	Tile 如何分配到 TP/EP/SP/DP 设备	Collective 算法、并行维度、拓扑

同一个 GEMM，不同 Execution Plan

BM=128/BN=128/BK=32 与 BM=64/BN=128/BK=64 的数学结果相同；2-stage 与 4-stage、M-major 与 N-major、1 CTA/SM 与 2 CTA/SM 也不会改变 FLOPs，却可能明显改变延迟。TileSight 就是要看见这些差别。

03 第一层：把 Tile 变成资源时间向量

每个 Tile Action 被映射为一组“在各硬件流水线上占用多长时间”的分量，而不是一个笼统的利用率。论文的硬件资源向量可概括为：

$$u(o) = \{ t_{TC}, t_{CUDA}, t_{SFU}, t_{TMEM}, t_{SMEM}, t_{L1.5}, t_{L2}, t_{DDR}, t_{Net} \}$$

资源维度	典型工作	为何必须单独建模
Tensor Core	MMA / WGMMMA / tcgen05.mma	与标量 CUDA Core 可并行，吞吐和启动特性不同
CUDA Core	地址计算、FMA、归约、softmax 标量部分	融合 Attention 中常与 Tensor Core 交错
SFU	exp、特殊函数	softmax / activation 可能形成独立瓶颈
TMEM	Blackwell tensor memory 数据路径	B200/SM100 的新层级，不能等同普通 SMEM
SMEM	片上 Tile staging、双/多缓冲	带宽、bank conflict、容量共同影响 pipeline
L1.5 / LRC	B200 双 die 等中间缓存层	有效带宽与容量不同于 L2/HBM
L2	跨 CTA / SM 的数据复用	对 Raster、Swizzle、persistent schedule 高度敏感
DDR / HBM	最终未命中流量	决定大多数 memory-bound kernel 的下界
Network	NVLink / IB 上的 Tile 远程访问	需要与本地 compute/memory 一起进入 overlap

Placement: 统一 Fusion 与 Distribution

Placement 是论文中非常漂亮的设计。若中间张量保留在 Register、TMEM 或 SMEM，下游算子可以直接消费，不产生 HBM store/load，这就是算子融合；若生产者和消费者位于不同 GPU，同一个张量访问会新增 Net 时间，这就是分布式通信。二者不需要两套互不相干的模型。

统一视角

Fusion = 数据没有离开本地片上 Placement；Distribution = 数据源 Placement 位于远端设备。最终都转成资源时间向量，再交给同一个流水线包络组合。

这一层比普通 Roofline 细的关键在于：同一资源上的工作必须串行累计，而不同资源可能重叠。例如 HBM Load 与 Tensor Core MMA 在稳态中可以并行；但两个都占用 Tensor Core 的动作不能同时获得满吞吐。

04 第二层之一：Prologue - Steady - Epilogue 流水线

Prologue - Steady - Epilogue : Tile 流水线为什么比 Roofline 更细

示意：每个 K-iteration 包含 Load、MMA、Store，不同硬件资源可重叠，同一资源上的工作串行累加

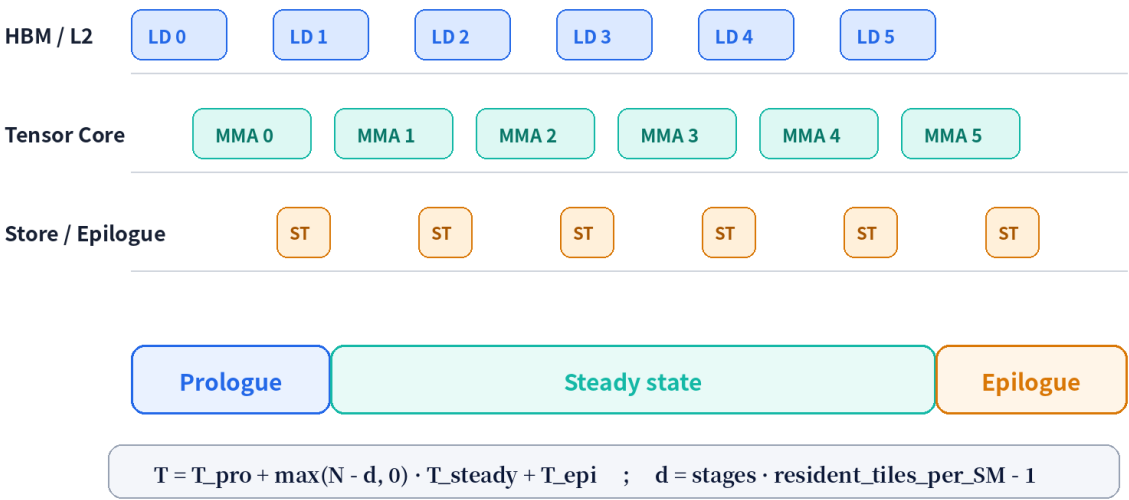


图2 Tile 流水示意。时间数字仅用于解释，非论文实测。

$$T = T_{\text{pro}} + \max(N - d, 0) \cdot T_{\text{steady}} + T_{\text{epi}}$$

有效流水深度: $d = \text{stages} \cdot \text{resident_tiles_per_SM} - 1$

这条公式把总时间拆成填充、稳定重叠和排空三个阶段。Pipeline stage 并不是唯一变量：一个 SM 能同时驻留多少个 CTA，也会增加可交错的 Tile 数。寄存器或 SMEM 使用过大，虽然单 CTA 的 stage 更多，却可能把 resident CTA 从 2 降到 1，最终有效流水深度反而下降。

一个直观数值例子

每轮 K Tile：Load 80 ns，MMA 100 ns，Store 20 ns。

完全串行：每轮约 200 ns。

如果 Load/Store 与 MMA 在不同流水线上充分重叠，稳态约为 $\max(100 \text{ ns memory-side}, 100 \text{ ns tensor-side}) = 100 \text{ ns}$ 。

但若只有 2-3 轮迭代，Prologue 与 Epilogue 占比很高，真实时间仍更接近串行。

为什么小 batch decode 难预测

大 GEMM 有足够多迭代摊薄填充与尾部；小 M / batch=1 kernel 可能只有极少 CTA 和极短循环，Launch latency、Tensor Core underfill、指令依赖与 wave tail 会占主导，而这些正是吞吐级模型的弱项。

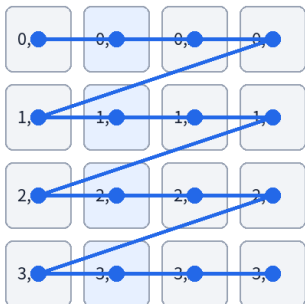
论文递归使用同一个 envelope：先组合 Tile 内 Action，再组合 K-loop，再组合 CTA Wave，最后组合计算 Stage 与通信 Stage。这个递归性让模型能够描述 FlashAttention、MLA 以及 AllGather+GEMM 等复杂融合。

05 第二层之二：Tile 级缓存复用距离

Tile Reuse Distance：同样的 FLOPs / Bytes，遍历顺序仍可改变真实 HBM 流量

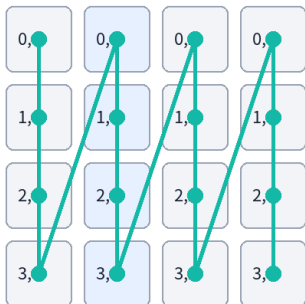
示意：同一个 B Tile 沿 M 维复用。再次访问它之前插入的不同 Tile 越少，L2 命中概率越高。

Row-major / 复用间隔较长



相同 B Tile 的再访问间隔：较大

Panel traversal / 复用间隔较短



相同 B Tile 的再访问间隔：较小

调优含义：Raster / Swizzle / Persistent CTA 可在不改变数学计算的情况下改变 L2 命中率与 DDR 时间。

图 3 Tile 遍历顺序对复用距离的影响。图为概念重绘。

传统缓存模拟器逐 Cache Line 追踪访问。对 GPU Tile 程序，这会迅速膨胀：一个 8 KB Tile 就含有数十条 128 B Cache Line。TileSight 改为追踪 Tile-sized Tensor Block，并利用 schedule 的规则性推导复用。

Reuse Key

$$\text{key}(\mathbf{x}, \mathbf{R}) = \text{Linearize}(\mathbf{x}_d \mid d \in \mathbf{R})$$

把 Tile 坐标投影到“非复用维度”，相同 key 表示访问同一逻辑 Tensor Tile。

以 GEMM 为例：A Tile 通常沿 N 维复用，B Tile 沿 M 维复用。M-major、N-major、panel traversal、Z-order、swizzle 与 persistent CTA 会改变相同 key 再次出现前插入了多少其他 Tile。这个数量就是 Tile Reuse Distance D_T 。

从复用距离到命中概率

TileSight 使用随机距离缓存模型 SDCM，将 D_T 、associativity 与有效 Cache 容量映射为命中概率，再逐级得到 L1.5、L2 与 DDR 流量。对 B200 一类存在 L1.5/LRC 的平台，会按缓存层级串联计算。架构的“有效容量”来自一次性带宽 sweep 校准，而非只使用标称容量。

为何能指导 Swizzle

换一个 Swizzle 不改变 FLOPs 和理论 Bytes，却可能让同一 B Tile 更早被再次访问： D_T 下降 → L2 hit 上升 → 实际 DDR Bytes 下降 → t_{DDR} 下降。普通 Roofline 不能表达这条链。

缓存模型的强项与关键反例

论文在 4,680 个 persistent GEMM case 上报告：A100/H200/B200/B6000 的 L2 命中率平均绝对误差分别为 1.46、0.88、1.05、0.78 个百分点。[1, Sec. 5.3] 这说明 Tile 粒度复用距离在大多数规则 schedule 中非常有效。

不能被平均数掩盖的 Outlier

H200 上 $M=N=8192$ 、 $K=28672$ 的深 K GEMM：TileSight 预测 L2 hit 82%，实测 43%。原因是模型假设 SM 近似同步推进，真实 SM 会失步并同时处理不同 K slice，使并发工作集远大于 lockstep 估计。[1, Sec. 5.3]

这类严重 outlier 对自动调优尤其危险：即使总体 MAE 很小，也可能把某个错误预测的配置排到第一。工程上应为深 K、persistent、跨 SM skew 明显的区域设置更保守的置信度或强制真机验证。

06 第三层：从单卡 Tile 推到多卡集群

远程 Tensor Tile 被展开成逻辑交换三元组：

(s, d, b) = (source device, destination device, bytes)

Collective 算法被分解为有序 Stage；每个 Stage 中的交换根据 NVLink、NVSwitch、InfiniBand 等拓扑路由，再以 α - β 模型估算启动延迟与拥塞链路上的序列化成本。

$$T_k = \max_{(s,d,b)} [\sum_{l \in P_{sd}} \alpha_l] + \max_l [\beta_l \cdot B(l,k)]$$

第一项近似路径启动/跳数成本，第二项由最拥塞链路决定。

得到的通信时间写入资源向量的 Net 维度，因此通信不是简单地追加在计算之后，而是可以与 Tensor Core、CUDA Core、HBM 等资源一起进入流水线 overlap。

场景	TileSight 中的表示	能够分析的核心问题
AllGather + GEMM	远程输入 Tile + 本地 GEMM Action	通信是否被 GEMM 稳态隐藏
GEMM + ReduceScatter	输出 Tile 分段归约与发送	Epilogue 与网络 Stage 的重叠
Ulysses Attention	Attention Tile + 多阶段 All-to-All	局部 FA pipeline 与 A2A 拥塞
Tensor Parallel	权重/激活 Tile 的设备 Placement	AllReduce/AG/RS 成本和拓扑
Expert Parallel	Token/Expert Tile 的远程放置	All-to-All 与 Expert GEMM 的 overlap

边界

它是 Stage/Link 粒度模型，不是 packet-level 网络模拟器。真实系统中的协议开销、队列、NIC contention、runtime scheduling 和长尾仍可能超出 α - β 近似。

07 实验结果：数字说明了什么

评测对象	规模 / 平台	TileSight 结果	应如何解读
单卡 Tensor-Core GEMM	703 个 BF16/FP16 shape; A100/H200/B200/B6000	12.35% pooled MAPE	足以做 schedule ranking，不是微秒级绝对延迟保证
MI210 GEMM	CK，无显式 raster/swizzle 控制	23.4% MAPE	AMD 仍可迁移，但误差明显高于新 NVIDIA 平台
FA-3 sanity check	H100，单个融合配置	5.73 ms vs NCU 5.58 ms，约 2.7%	证明可处理非 GEMM，但单点不能代表所有融合算子
L2 命中率	4,680 个 persistent GEMM	0.78-1.46 pp MAE	缓存抽象强，但深 K skew 有严重 outlier
纯 Collective	152 case；H200×8/B200×8	12.22% wMAPE	拓扑校准有效；baseline 条件不完全公平
融合计算通信	152 case	正文 14.83% wMAPE；摘要 16.18%	v1 数字口径需作者解释

评测对象	规模 / 平台	TileSight 结果	应如何解读
vLLM decode	166 个 dense/MoE、单/多节点配置	13.52% wMAPE	系统级组合能力强，但极小 batch 仍是弱项
TileLang Cost Model	10 个 LLaMA-derived GEMM	Top 5% 达到 99.66% 穷举最优	这是最有工程价值的结果

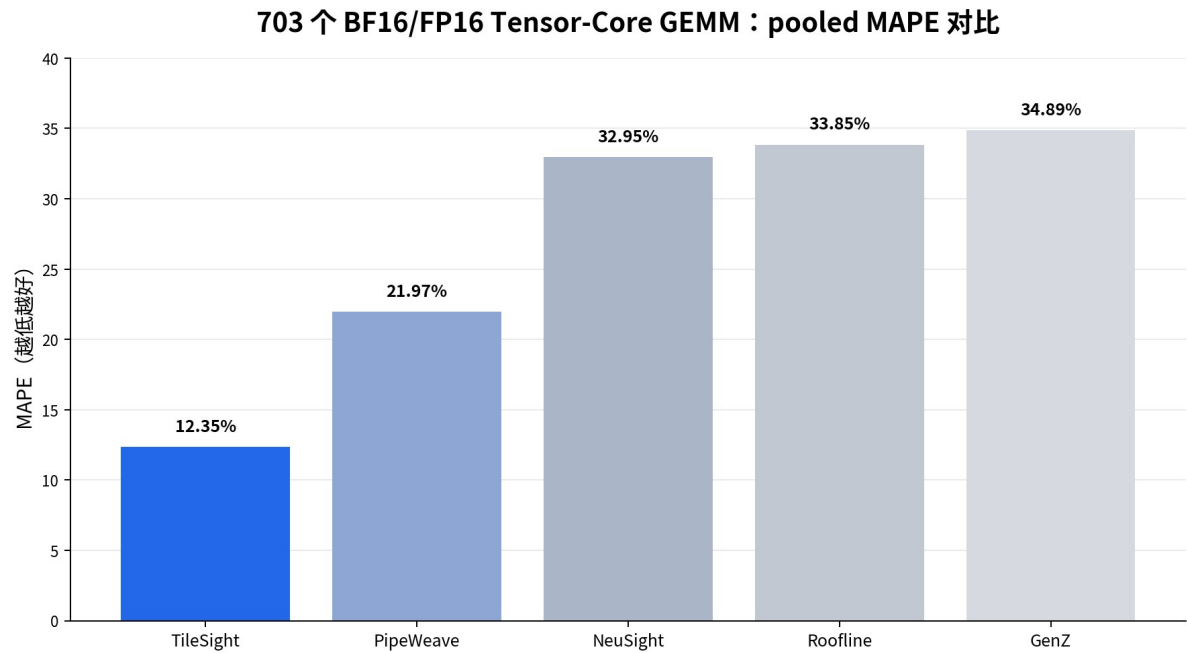


图4 单卡 GEMM pooled MAPE。数据取自论文 Figure 5 与 Sec. 5.2。

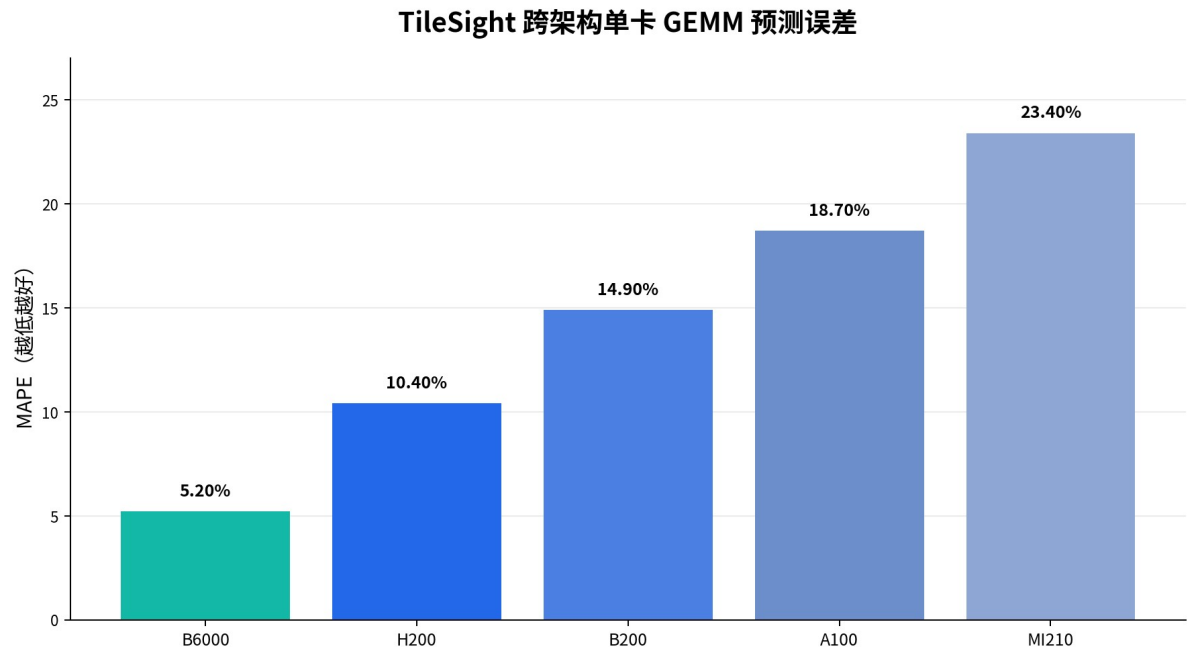


图5 TileSight 在不同架构上的 GEMM MAPE。MI210 为论文单独报告。

最重要的实验结论不是 “12.35%”

如果把 TileSight 当成离线预测器，10%-20% 的误差并不算足以替代 benchmark；但如果把它当作候选筛选器，Top 5% 保留 99.66% 最优性能的结果非常有价值。编译器需要的往往不是 “7.31 μ s 的精确预测”，而是 “几千个 schedule 中哪几十个值得跑”。

跨架构迁移

NeuSight 在 A100 上略优，是因为 A100 位于其训练分布；到了 H200/B200/B6000，TileSight 的白盒校准更稳定。论文真正想证明的是：把结构编码进模型，比依赖某一代硬件的训练数据更容易迁移。

08 批判性审视：论文 v1 最需要澄清的地方

8.1 拓扑序目标函数在当前写法下似乎与顺序无关

论文把融合 kernel 的合法 Action 顺序记为 σ ，并给出稳态时间：

$$T_{\text{steady}}(\sigma) = \max_r \sum_{(o \in \sigma)} u_r(o)$$

但只要 σ 是同一组 action 的排列，对固定资源 r ， $\sum u_r(o)$ 与顺序无关；因此再对所有合法拓扑序取 min，按这个公式不会得到不同结果。要让 order 真正影响时间，模型需要显式的阶段边界、依赖导致的 bubble、有限 buffer、资源占用区间或 order-dependent state。

DeepStack 中 source-visible 的 `overlap_analysis.py` 将可见实现描述为枚举合法拓扑序；HardwareUsage 的核心聚合又是同一资源求和、不同资源取 max。[2][4] 从公开代码片段看，简单聚合仍难以产生顺序差异。这里需要保持谨慎：仓库可能存在其他路径或未展示状态，但论文 v1 的数学表达本身确实不充分。

影响范围

这不推翻 Tile 资源向量、缓存和分布式模型，但会削弱论文关于“通过拓扑序搜索精确建模复杂融合 kernel 内部调度”的论证。

8.2 SDCM 正态近似公式很可能有排版错误

论文式 (8) 出现绝对值形式：

$$P(\text{hit} \mid D_T) = 1 - Q(|A - 1 - \mu| / \sqrt{\sigma^2})$$

若 Q 是标准正态互补 CDF，则 $1 - Q(|x|) = \Phi(|x|) \geq 0.5$ ，这与大量低于 50% 的 Cache Hit 预测不相容。公开 `sdcm.py` 实际使用带连续性修正的有符号输入：

$$\Phi((A - 1 + 0.5 - \mu) / \sigma)$$

代码形式在数学上合理，因此更可能是论文排版/符号错误，而不是实现错误。[3]

8.3 “No kernel profiling/training” 不等于零测量

TileSight 不需要为目标 kernel 收集 trace，也不需要训练 ML 模型，但每代架构仍需一次性 microbenchmark：Tensor Core/CUDA Core/SFU 吞吐、SMEM/TMEM/L2/HBM 带宽、有效缓存容量、网络 α/β 、占用上限等。论文表 1 也明确说明这类校准约需数分钟。[1, Table 1]

8.4 分布式 baseline 并非完全同条件

PipeWeave 没有原生 H200-NVL/B200 Collective backend，论文回退到 H800 模型，因此无法反映 NVLink4/5 差异。TileSight 的胜出部分来自模型更合理，也部分来自 baseline 被迫跨架构外推。结论仍有价值，但不宜把误差差距全部归因于算法优劣。

8.5 摘要与正文的 fused-kernel 数字不一致

摘要报告 fused distributed kernels 为 16.18% wMAPE，Sec. 5.4 正文报告 14.83% wMAPE。可能对应不同筛选范围或聚合口径，但 v1 没有清楚解释。引用时应同时注明两个位置，避免只取更漂亮的数字。

8.6 明确的适用边界

不建模数据依赖控制流和高度不规则内存访问。

不建模指令级编译决策、真实寄存器分配、SASS 依赖链。

不掌握未公开的 warp/CTA scheduling 和 closed-source runtime 行为。

不做 packet-level 网络仿真。

对 B200 多 die 的 SM-HBM affinity、跨 die skew 等仍需增强。

论文明确指出 small-batch decode attention 等 latency-bound case 是弱项。[1, Sec. 7]

综合判断

方向和抽象很强，缓存部分最扎实；作为 Cost Model 的证据比作为“精确模拟器”的证据更充分。当前 v1 值得跟进，但公式严谨性和复杂 DAG 调度部分需要等待修订或作者解释。

09 与你当前 Full-K deterministic GEMM 的直接关系

针对小 M Full-K GEMM：TileSight 最适合做候选筛选，而不是绝对延迟预言

目标形状：A[M,2048] × B[2048,128] → C[M,128]，M = 1 / 2 / 4 / 8 / 12



图 6 小 M Full-K GEMM 的主要权衡与推荐用法。

$$A[M, 2048] \times B[2048, 128] \rightarrow C[M, 128], \quad M \in \{1, 2, 4, 8, 12\}$$

对于这个形状，TileSight 可以把 kernel 描述为二维输出 TileGrid，并在每个 CTA 内执行 2048/BK 轮 K-loop：Load A/B → MMA → Accumulate → Store C。它能够同时看到并行 CTA 数、缓存复用、stage、驻留度和 wave tail。

你的优化旋钮	TileSight 对应抽象	主要影响
BM / BN	TileGrid 与 Wave	CTA 数、Tensor Core 填充、A/B 重复加载、尾波
BK	K-loop 单次 Tile footprint	循环次数、SMEM/寄存器压力、pipeline 粒度
stages	Pipeline depth	Load-MMA overlap；同时影响 SMEM 与 occupancy
block threads / warps	Resident tiles per SM	有效流水深度、调度并行性
raster / swizzle	Traversal + Reuse Distance	A/B Tile 的 L2 局部性
Register/SMEM staging	Placement	是否产生 HBM 中间流量、bank conflict 风险
Full-K / Split-K	DAG 与跨 CTA reduction	并行度、额外归约、确定性与通信

核心权衡 1: Tile 越大，复用越好，但 CTA 越少

当 BN=128 时，N 方向只有一个 CTA，A 不需要跨 N-CTA 重复加载；但 M 很小时，整个 kernel 可能只有 1-2 个 CTA。把 BN 减到 32 会制造 4 个 N-CTA，提高并行度，却让同一 A Tile 被重复访问。BM 同理：更小 BM 增加 M 方向 CTA，却重复加载 B。

核心权衡 2: BK / stages 不是越大越好

更大 BK 减少 K-loop 次数和边界开销，但抬高 SMEM 与 accumulator/register footprint，可能把 resident CTA 从 2 降到 1。此时公式中的 $d = \text{stages} \times \text{resident CTA} - 1$ 可能下降，稳态 overlap 反而变差。

核心权衡 3: Full-K 的确定性优势与并行度代价

Full-K 让一个 CTA 按固定 K 顺序完成完整输出归约，不需要 Split-K partial sum 和跨 CTA reduction，因而更容易维持 bitwise reproducibility。TileSight 能估算“放弃 Split-K 后损失了多少 CTA 并行度、Tensor Core 利用率和 Wave 效率”，但它不会证明浮点确定性；确定性仍需从归约顺序、atomic、编译器重排和 kernel launch 行为验证。

推荐的落地流程

1. 枚举 BM/BN/BK/stages/warps/raster/swizzle，并先做资源合法性过滤。
2. 用 TileSight 或类似分析模型给候选排序，保留 Top 5%-10%，同时为深 K/persistent 高风险区保留若干探索样本。
3. 在目标 H200/B200 上用 CUDA Event 或框架 benchmark 进行稳定测量，覆盖 warmup、不同 M 与多轮重复。
4. 用 NCU 核对 Tensor Core 活跃度、L2 hit、HBM 流量、SMEM、occupancy、wave count 与 launch 间隙。
5. 对最后几个候选查看 SASS/寄存器数和依赖链，单独处理模型未覆盖的微秒级固定开销。
6. 确定性验证与性能验证分开：跨运行、多 stream、不同 batch 顺序、不同图捕获方式都要比较 bitwise 输出。

最重要的提醒

M=1/2/4/8/12 正好属于论文承认的 latency-bound 区域。TileSight 可以提供方向和 prior，但最终冠军必须由真机 benchmark 决定。

10 对 MLA、MoE 与 vLLM/SGLang 的意义

MLA / Attention

MLA decode 中，KV Cache Tile 可能沿 attention head 复用。TileSight 能把 Q/K/V Load、两次 GEMM、Rescale、Softmax、Store 组成 Action DAG，并让 Head 遍历顺序直接影响 Reuse Distance；同时，Register/TMEM/SMEM Placement 决定中间结果是否落 HBM。这正适合比较不同 fused MLA schedule。

MoE / Expert Parallel

对规则、已知 token 分布的 MoE，模型可以表达 Expert Weight 位于哪张 GPU、Token activation 是否触发 All-to-All，以及 Expert GEMM 能否隐藏通信。但真实 MoE 的 token-to-expert route 是数据依赖的，还会出现热门 Expert、负载不均、capacity overflow、All-to-All 长尾与动态 batch。

与你的 Router Replay / R3 工作的交点

TileSight 可作为“给定路由结果后的性能模型”，估算某一 expert assignment 产生的计算与通信成本；它本身不会判断路由是否最优，也不会自然建模不同 token 分布的随机性。更完整的方案需要把 Router Replay 产生的 expert histogram、每卡 token 数和通信矩阵输入到模型。

vLLM / SGLang 系统级组合

论文将单算子预测、缓存流量、Wave 与 Collective 组合到端到端 vLLM decode，说明 Tile 级 cost 可以向上服务系统模拟。不过真正线上 serving 还包含 continuous batching、scheduler、KV cache allocator、prefix hit、CPU launch、CUDA Graph、动态 shape 和网络抖动。TileSight 主要覆盖其中的 GPU kernel 与跨卡数据路径。

场景	TileSight 很有用	还需要补充
MLA fused kernel	比较 Tile shape、head traversal、TMEM/SMEM Placement	真实寄存器分配、SASS 依赖、small-batch 固定开销
MoE Expert GEMM	规则 shape 的计算/缓存/occupancy 预测	动态 token 分布与 expert imbalance
EP All-to-All	给定通信矩阵后的拓扑路由与 overlap	队列、runtime、长尾、协议细节
vLLM decode	算子与通信的白盒组成	scheduler、continuous batching、KV 管理与 host overhead
Compiler autotune	快速筛掉 90%-95% 差候选	最终 benchmark 与 online feedback

11 如何在面试中讲清这篇论文

90 秒版本

TileSight 解决的是 GPU 编程已经 Tile 化，但性能建模仍停留在 Kernel 级的问题。它把每个 Tile 转成 Tensor Core、CUDA Core、SFU、TMEM、SMEM、L2、HBM 和网络的资源时间向量，通过 Prologue-Steady-Epilogue 建模流水，通过 Tile Reuse Distance 推导多级缓存命中，再用 Placement 把融合与跨卡通信统一起来。它的优势是白盒、跨架构、能做编译器 Cost Model；弱点是吞吐级近似，无法完整覆盖指令调度、寄存器分配和 small-batch latency。论文 v1 的拓扑序公式与 SDCM 公式也有需要作者澄清的地方。

面试官继续追问时的四个抓手

- 1. 为什么比 Roofline 强：相同 FLOPs/Bytes 的 schedule 可以因 pipeline、occupancy、cache 与 placement 不同而性能不同。
- 2. 为什么能跨架构：结构由 Tile Plan 给出，硬件差异进入一次性 throughput/bandwidth/cache/network 校准。
- 3. 为什么缓存模型重要：Swizzle/Raster 改变 Reuse Distance，从而改变真实 DDR traffic。
- 4. 为什么仍需 benchmark：10%-20% MAPE 适合 ranking，不足以替代微秒级真机测量；尤其 small-M decode。

个人评价

维度	评分	理由
选题与方向	9.0 / 10	精准抓住 Tile 编程与性能分析之间的抽象错位
核心抽象	9.0 / 10	资源向量、Placement、分层组合统一且工程感强
缓存模型	9.0 / 10	Schedule-sensitive，速度与准确度平衡出色
分布式扩展	8.0 / 10	统一进入 Net 维度很漂亮，但仍是 α - β Stage 粒度
实验说服力	7.5 / 10	覆盖较广；部分 baseline、公示口径与单点 fused 验证仍可加强
数学/实现严谨性	6.5 / 10	v1 拓扑序与 SDCM 公式需要澄清
Compiler Cost Model 价值	9.0 / 10	Top 5% 达到 99.66% 最优是最强工程证据
小 M Decode 绝对预测	6.0 / 10	论文自己承认 latency-bound 是当前短板

最后带走三点

- 1) 性能模型的抽象层级，应与程序员真正修改的调度层级一致；
- 2) Tile shape、pipeline、occupancy、cache reuse 与 placement 必须联合分析；
- 3) 对小 M Full-K GEMM，分析模型负责缩小搜索空间，真机 benchmark 才负责定冠军。

12 参考资料

[1] 论文原文 (arXiv v1)

<https://arxiv.org/pdf/2607.22432>

[1a] arXiv 摘要与元数据页

<https://arxiv.org/abs/2607.22432>

[2] DeepStack 中的 TileSight source snapshot

<https://github.com/tile-ai/DeepStack/tree/ae/src/tilesight>

[3] SDCM 实现: `sdcm.py`

<https://raw.githubusercontent.com/tile-ai/DeepStack/refs/heads/ae/src/tilesight/tilesight/util/sdcm.py>

[4] 融合流水与拓扑序实现: `overlap_analysis.py`

https://github.com/tile-ai/DeepStack/blob/ae/src/tilesight/tilesight/fused_op_pipeline_wave/overlap_analysis.py

术语速查

术语	含义
MAPE / wMAPE	平均绝对百分比误差 / 加权平均绝对百分比误差
Tile	GPU kernel 中规则的张量数据块与其对应工作单元
CTA	Cooperative Thread Array, 通常对应 CUDA block
Placement	张量或 Tile 所在的存储层级或设备
Reuse Distance	两次访问同一对象之间出现的不同对象数量
SDCM	Stochastic Distance-based Cache Model
Prologue / Steady / Epilogue	流水线填充 / 稳态 / 排空阶段
α - β Model	以固定启动延迟 α 和单位字节成本 β 近似通信时间
Full-K	单个 CTA 覆盖完整 K 维归约, 避免跨 CTA partial reduction

注：本文为面向 AI Infra / GPU Kernel Optimization 的独立研读报告，不替代论文作者的正式解释。对 v1 公式和代码的批判性观察应以未来修订、正式发表版本或作者回复为最终依据。